

Análise Estatística de Métricas de Código-Fonte do Framework Neo4j

Igor Mariano¹, Felipe¹

Curso de Graduação – Engenharia de Software
{igor.mariano, felipe}@aluno.edu.br

Abstract.

This paper presents a statistical analysis of source code metrics extracted from the Neo4j framework, a graph database system written in Java. Using the GitHub Bug Dataset (version 1.1), we analyze two granularity levels — class and file — encompassing 7,917 and 4,261 instances respectively. The study applies descriptive statistics, normality tests (Shapiro-Wilk and Kolmogorov-Smirnov), correlation analysis using Pearson coefficients, outlier detection via the IQR method, and logistic regression modeling. Results reveal strong correlations among size and complexity metrics (e.g., $\text{LOC} \times \text{LLOC}$: $r = 0.99$; $\text{WMC} \times \text{LOC}$: $r = 0.89$), highly right-skewed distributions for all metrics, and a severe class imbalance in the bug variable ($< 0.15\%$ positive). The logistic regression models achieved high accuracy but failed to predict defective instances, highlighting the challenges of fault prediction with imbalanced data.

Resumo.

Este artigo apresenta uma análise estatística de métricas de código-fonte extraídas do framework Neo4j, um sistema de banco de dados em grafo escrito em Java. Utilizando o GitHub Bug Dataset (versão 1.1), analisamos dois níveis de granularidade — classe e arquivo — com 7.917 e 4.261 instâncias respectivamente. O estudo aplica estatística descritiva, testes de normalidade (Shapiro-Wilk e Kolmogorov-Smirnov), análise de correlação com coeficientes de Pearson, detecção de outliers pelo método IQR e modelagem por regressão logística. Os resultados revelam correlações fortes entre métricas de tamanho e complexidade (e.g., $\text{LOC} \times \text{LLOC}$: $r = 0,99$; $\text{WMC} \times \text{LOC}$: $r = 0,89$), distribuições fortemente assimétricas à direita para todas as métricas e um desbalanceamento severo na variável de bugs ($< 0,15\%$ positivos). Os modelos de regressão logística alcançaram alta acurácia, porém falharam em prever instâncias defeituosas, evidenciando os desafios da predição de falhas em dados desbalanceados.

1 Introdução

A análise quantitativa de métricas de código-fonte constitui uma abordagem fundamental na Engenharia de Software para compreender a qualidade, a complexidade e a propensão a defeitos de sistemas de software [5]. Métricas orientadas a objetos, como as propostas por Chidamber e

Kemerer [2], permitem avaliar características estruturais de projetos Java e têm sido amplamente utilizadas em estudos empíricos de predição de falhas [3, 1].

O framework Neo4j é um sistema de banco de dados em grafo de código aberto, amplamente utilizado em aplicações que demandam modelagem de relacionamentos complexos. Por sua relevância no ecossistema Java e pelo volume considerável de código-fonte, constitui um caso de estudo relevante para análise de métricas de software.

Este trabalho tem como objetivo aplicar técnicas de estatística descritiva e inferencial sobre datasets de métricas de código-fonte do framework Neo4j, extraídos do GitHub Bug Dataset versão 1.1 [9]. A análise contempla: (i) medidas de tendência central e dispersão; (ii) construção de gráficos para visualização das distribuições; (iii) detecção de outliers; (iv) testes de normalidade; (v) análise de correlação entre métricas; e (vi) ajuste de um modelo de regressão logística para predição de defeitos.

O repositório completo com os scripts, dados e figuras está disponível em: <https://github.com/igormariano/AP1-Projeto-ES-Neo4j>.

2 Metodologia

2.1 Fonte de Dados

Os datasets utilizados foram obtidos do GitHub Bug Dataset versão 1.1 [9], que contém métricas de código-fonte extraídas de projetos Java hospedados no GitHub. Para o framework Neo4j, foram selecionados os datasets mais recentes disponíveis, organizados em dois níveis de granularidade:

- **Dataset Class-level:** 7.917 instâncias com 113 atributos, incluindo métricas de complexidade, acoplamento, coesão, herança, tamanho, documentação e warnings de análise estática.
- **Dataset File-level:** 4.261 instâncias com 12 atributos, incluindo complexidade ciclomática (McCC), linhas lógicas de código (LLOC) e métricas de histórico de modificações.

Nenhum dos datasets apresentou valores ausentes.

2.2 Seleção de Variáveis

A partir dos 113 atributos originais do dataset Class-level, foram selecionadas 19 variáveis numéricas para análise, com base nos seguintes critérios: relevância estatística, semântica da métrica, alinhamento com os requisitos da análise e ausência de redundância excessiva.

A Tabela 1 apresenta as variáveis selecionadas e suas justificativas.

Tabela 1: Variáveis selecionadas para análise (Class-level)

Variável	Categoria	Descrição
WMC	Complexidade	Complexidade ponderada dos métodos
CBO	Acoplamento	Acoplamento entre classes
RFC	Acoplamento	Resposta para a classe
LCOM5	Coesão	Falta de coesão dos métodos
DIT	Herança	Profundidade na árvore de herança
NOC	Herança	Número de filhos
LOC	Tamanho	Linhas de código
LLOC	Tamanho	Linhas lógicas de código
NOS	Tamanho	Número de statements
NM	Tamanho	Número de métodos
NL	Complexidade	Nível de aninhamento
NLE	Complexidade	Nível de aninhamento else-if
CC	Clones	Cobertura de clones
CD	Documentação	Densidade de comentários
CLOC	Documentação	Linhas de comentário
NA	Tamanho	Número de atributos
WarningInfo	Qualidade	Warnings informativos
WarningMajor	Qualidade	Warnings majoritários
N. de bugs	Alvo	Número de defeitos

As variáveis excluídas incluem: identificadores textuais (ID, Name, Path); metadados de localização (Line, Column); métricas totais redundantes com versões locais (prefixo T); métricas de clone granulares redundantes com CC; categorias individuais de regras PMD (cobertas pelos warnings agregados); e variáveis com variância zero (e.g., WarningBlocker = 0 em 100% das instâncias).

Para o dataset File-level, a coluna CLOC foi excluída por ser constante (todos os valores iguais a zero), mantendo-se 7 variáveis numéricas.

2.3 Ferramentas Utilizadas

A análise exploratória inicial foi conduzida em Python 3.12 com as bibliotecas `pandas` e `numpy`. A análise estatística definitiva, incluindo testes, gráficos e modelagem, foi realizada em R 4.5.2 [6], utilizando os pacotes `ggplot2`, `corrplot`, `PerformanceAnalytics` e `car`.

3 Resultados e Discussão

3.1 Medidas de Tendência Central

A Tabela 2 apresenta as medidas de tendência central para as principais variáveis do dataset Class-level.

Tabela 2: Medidas de tendência central — Class-level

Variável	Média	Mediana	Moda
WMC	7,39	3,00	1
CBO	5,64	3,00	2
RFC	9,84	5,00	1
LCOM5	1,58	1,00	1
LOC	59,70	25,00	7
LLOC	49,18	21,00	8
NOS	18,77	5,00	1
NM	8,61	4,00	1
DIT	0,98	1,00	1
CC	0,12	0,00	0
CD	0,05	0,00	0

Em todas as métricas de tamanho e complexidade, a média é substancialmente superior à mediana, indicando distribuições fortemente assimétricas à direita. A métrica LOC, por exemplo, apresenta média de 59,70 e mediana de apenas 25,00, evidenciando que a maioria das classes possui poucas linhas de código, enquanto um subconjunto menor apresenta valores elevados. A moda igual a 1 para WMC, RFC, NOS e NM indica que a maioria das classes no projeto Neo4j possui complexidade mínima.

Para o dataset File-level, observa-se padrão semelhante: McCC apresenta média de 6,06 e mediana de 2,00 (moda = 1), enquanto LLOC possui média de 102,74 e mediana de 55,00. A variável “Number of developer commits” apresenta moda de 5.350, correspondente ao desenvolvedor com maior número de contribuições.

3.2 Medidas de Dispersão

Tabela 3: Medidas de dispersão — Class-level

Variável	Amplitude	Variância	Desvio Padrão
WMC	325	193,25	13,90
CBO	161	55,57	7,45
RFC	293	256,19	16,01
LOC	3.207	13.360,81	115,59
LLOC	2.588	8.473,03	92,05
NOS	1.411	2.218,10	47,10
NM	161	183,94	13,56
DIT	6	0,99	0,99
CC	1	0,08	0,29

As métricas de tamanho apresentam as maiores amplitudes e variâncias. A variável LOC possui amplitude de 3.207 e desvio padrão de 115,59, o que é quase o dobro da média (59,70), confirmando a alta dispersão. O coeficiente de variação de LOC é de 1,94, indicando dispersão elevada.

A métrica DIT, por outro lado, apresenta baixa variabilidade (amplitude = 6, $\sigma = 0,99$), refletindo hierarquias de herança rasas no Neo4j.

No dataset File-level, destaca-se “Number of developer commits” com variância de 4.106.050,59 e desvio padrão de 2.026,34, decorrente da grande discrepância entre desenvolvedores.

3.3 Medidas de Posição Relativa e Boxplots

A análise de quartis e percentis reforça a assimetria observada. Para WMC no Class-level: $Q_1 = 1$, $Q_2 = 3$, $Q_3 = 8$, $P_{95} = 27$, com IQR = 7. Isso indica que 75% das classes possuem $WMC \leq 8$, enquanto o valor máximo é 325.

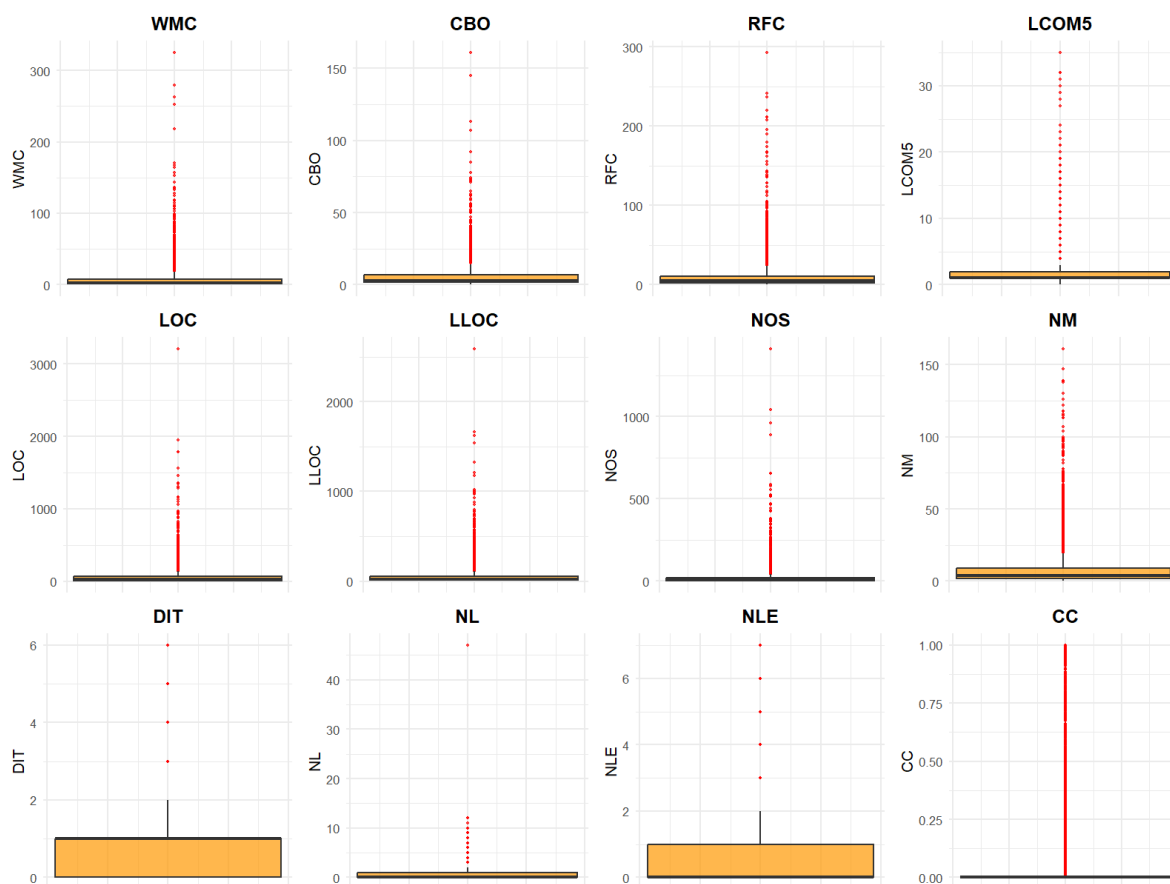


Figura 1: Boxplots das principais métricas — Class-level

A Figura 1 evidencia a presença massiva de outliers em todas as métricas, um padrão comum em projetos de software de grande porte. A Figura 2 apresenta os boxplots para o dataset File-level, onde padrão semelhante é observado.

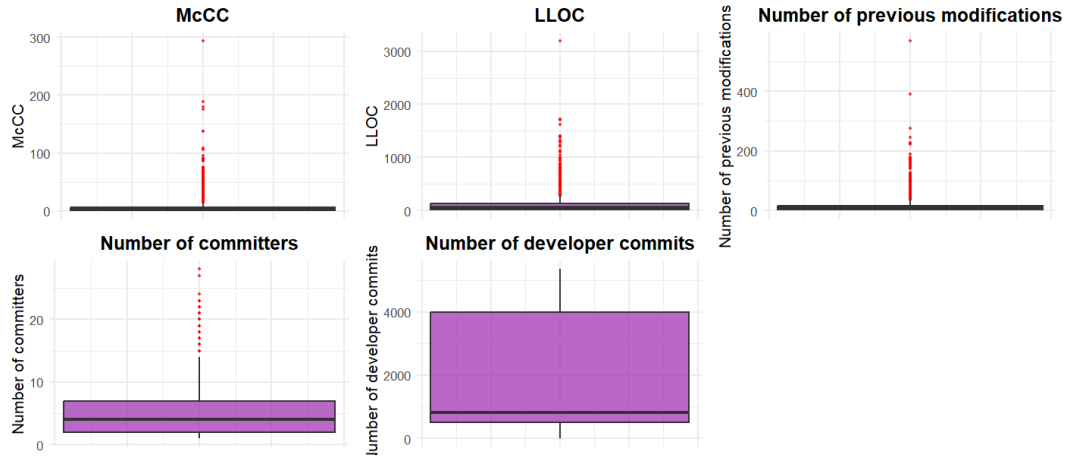


Figura 2: Boxplots das principais métricas — File-level

3.4 Visualizações Gráficas

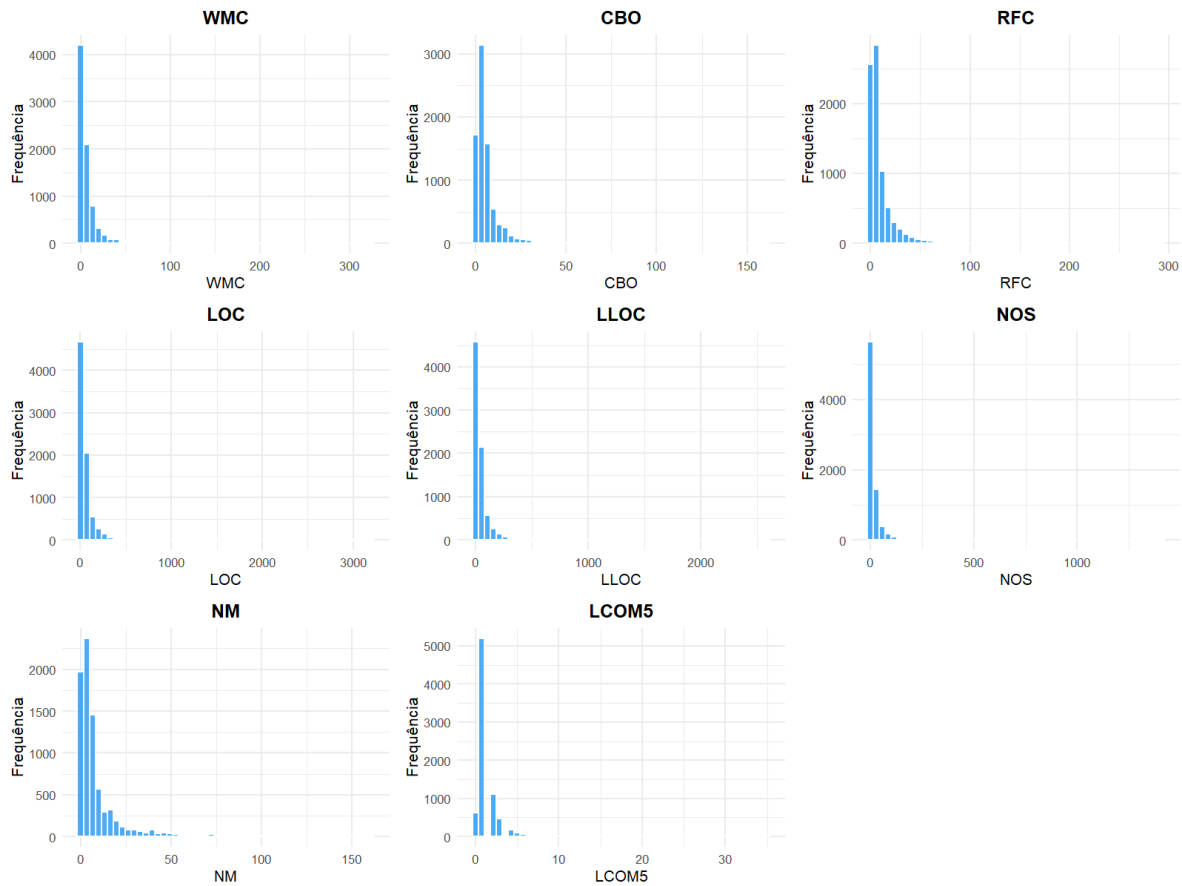


Figura 3: Histogramas das métricas Class-level

Os histogramas (Figura 3) confirmam que todas as variáveis seguem distribuições fortemente assimétricas à direita, com concentração massiva de valores baixos e caudas longas à direita. Esse

comportamento é esperado em métricas de software, onde a maioria dos componentes é simples e poucos concentram alta complexidade [7].

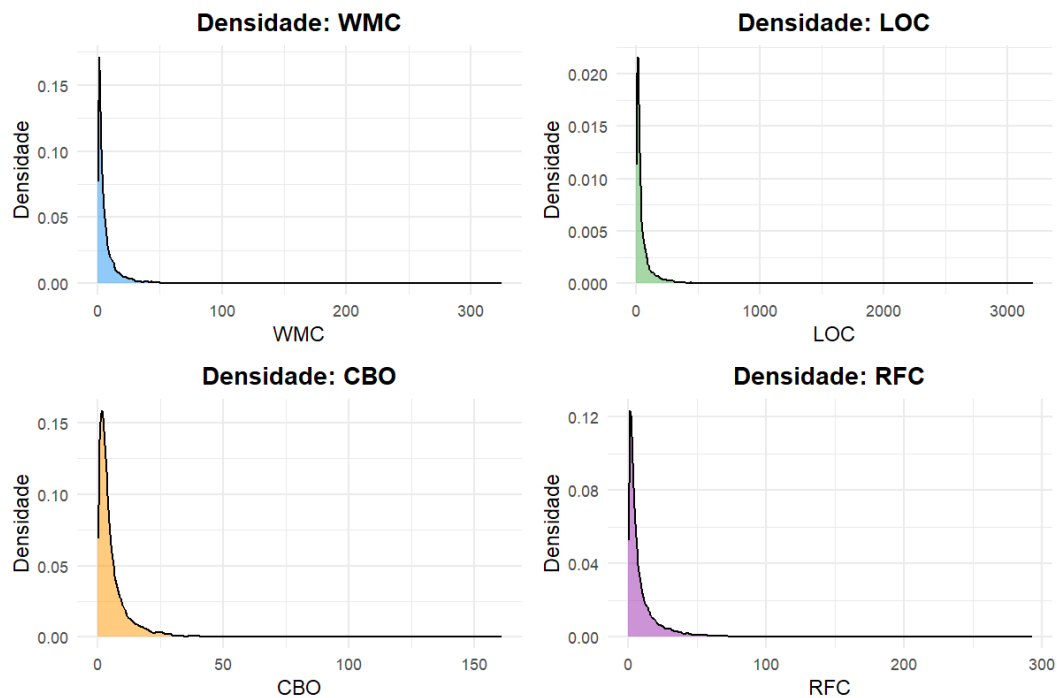


Figura 4: Gráficos de densidade para WMC, LOC, CBO e RFC

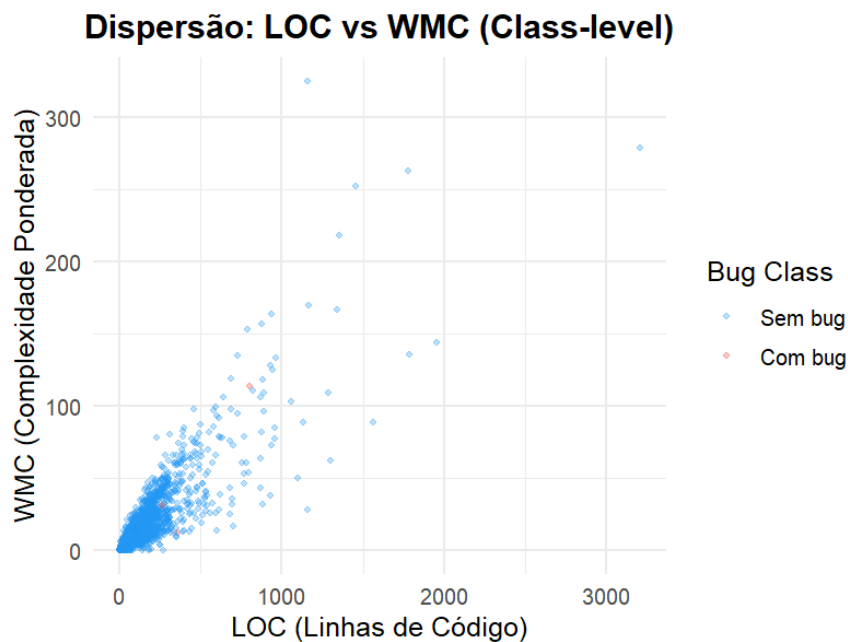


Figura 5: Dispersão: LOC vs WMC (Class-level)

A Figura 5 mostra a relação entre LOC e WMC, evidenciando uma correlação forte e

positiva. As instâncias com bugs (em vermelho) tendem a se posicionar em regiões de valores elevados, embora o número limitado de observações com defeitos (apenas 8) não permita conclusões robustas.

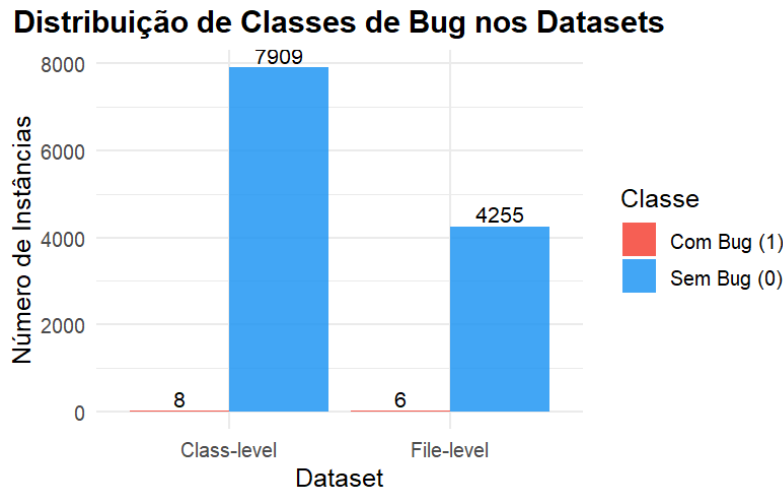


Figura 6: Distribuição de classes de bug nos dois datasets

A Figura 6 ilustra o severo desbalanceamento da variável alvo: apenas 8 classes (0,10%) e 6 arquivos (0,14%) apresentam bugs registrados.

3.5 Avaliação de Outliers

A Tabela 4 resume a detecção de outliers pelo método IQR ($Q_3 + 1,5 \times IQR$).

Tabela 4: Detecção de outliers pelo método IQR — Class-level

Variável	Outliers	% Total	Limite Sup.
WMC	706	8,92%	18,50
CBO	644	8,13%	14,50
RFC	759	9,59%	24,50
LOC	799	10,09%	140,00
LLOC	787	9,94%	112,50
NOS	840	10,61%	42,00
NM	866	10,94%	19,50
CC	1.780	22,48%	0,00
CLOC	1.155	14,59%	7,50

A proporção de outliers varia entre 5,86% (NA) e 22,48% (CC). A métrica CC (clone coverage) apresenta o maior número de outliers, pois a maioria dos valores é zero e qualquer valor positivo é classificado como outlier pelo método IQR. Para as métricas de tamanho (LOC, LLOC, NOS), aproximadamente 10% das instâncias são outliers, o que é consistente com a regra de Pareto aplicada a software: uma minoria de componentes concentra a maior complexidade [4].

Os outliers observados não foram removidos, pois representam classes legitimamente complexas do framework Neo4j, e removê-los distorceria a análise.

3.6 Testes de Normalidade

Para todas as variáveis de ambos os datasets, os testes de Shapiro-Wilk e Kolmogorov-Smirnov rejeitaram a hipótese nula de normalidade ($p < 10^{-50}$ em todos os casos). A Tabela 5 apresenta as estatísticas de teste para as principais variáveis.

Tabela 5: Testes de normalidade — Class-level (amostra de 5.000)

Variável	SW Stat	SW p	KS Stat	KS p
WMC	0,4214	$< 10^{-83}$	0,3081	$< 10^{-300}$
CBO	0,5874	$< 10^{-76}$	0,2246	$< 10^{-300}$
RFC	0,5158	$< 10^{-79}$	0,2786	$< 10^{-300}$
LOC	0,4244	$< 10^{-83}$	0,3113	$< 10^{-300}$
DIT	0,8267	$< 10^{-59}$	0,2520	$< 10^{-300}$
CC	0,4754	$< 10^{-81}$	0,4410	$< 10^{-300}$

A estatística de Shapiro-Wilk [8] varia entre 0,013 (Number of bugs) e 0,827 (DIT), sempre muito distante de 1 (normalidade perfeita). Esses resultados são esperados para métricas de software, que tipicamente seguem distribuições lognormais ou de Pareto [7]. A rejeição da normalidade justifica o uso de métodos não paramétricos ou transformações em análises subsequentes.

3.7 Análise de Correlação

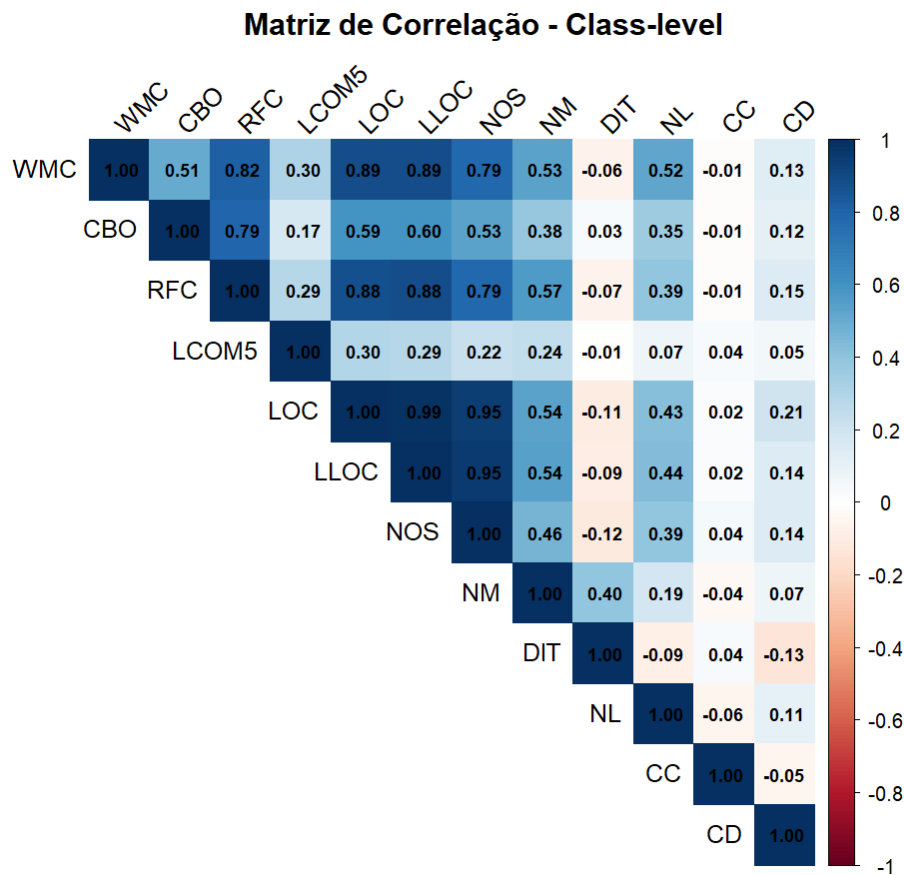


Figura 7: Heatmap de correlação — Class-level

A Figura 7 apresenta a matriz de correlação de Pearson para as principais métricas do Class-level. As correlações mais relevantes são:

Correlações fortes ($|r| \geq 0,7$):

- $LOC \times LLOC$: $r = 0,99$ — redundância quase total entre as duas métricas de tamanho;
- $LOC \times NOS$: $r = 0,95$ — tamanho total fortemente associado ao número de statements;
- $WMC \times LOC$: $r = 0,89$ — complexidade cresce com o tamanho;
- $WMC \times RFC$: $r = 0,82$ — classes complexas tendem a interagir com mais métodos;
- $CBO \times RFC$: $r = 0,80$ — acoplamento e resposta da classe são intimamente relacionados.

Correlações moderadas ($0,4 \leq |r| < 0,7$):

- $WMC \times CBO$: $r = 0,51$;

- WMC $\times$ NM: $r = 0,53$;
- CBO $\times$ LOC: $r = 0,59$.

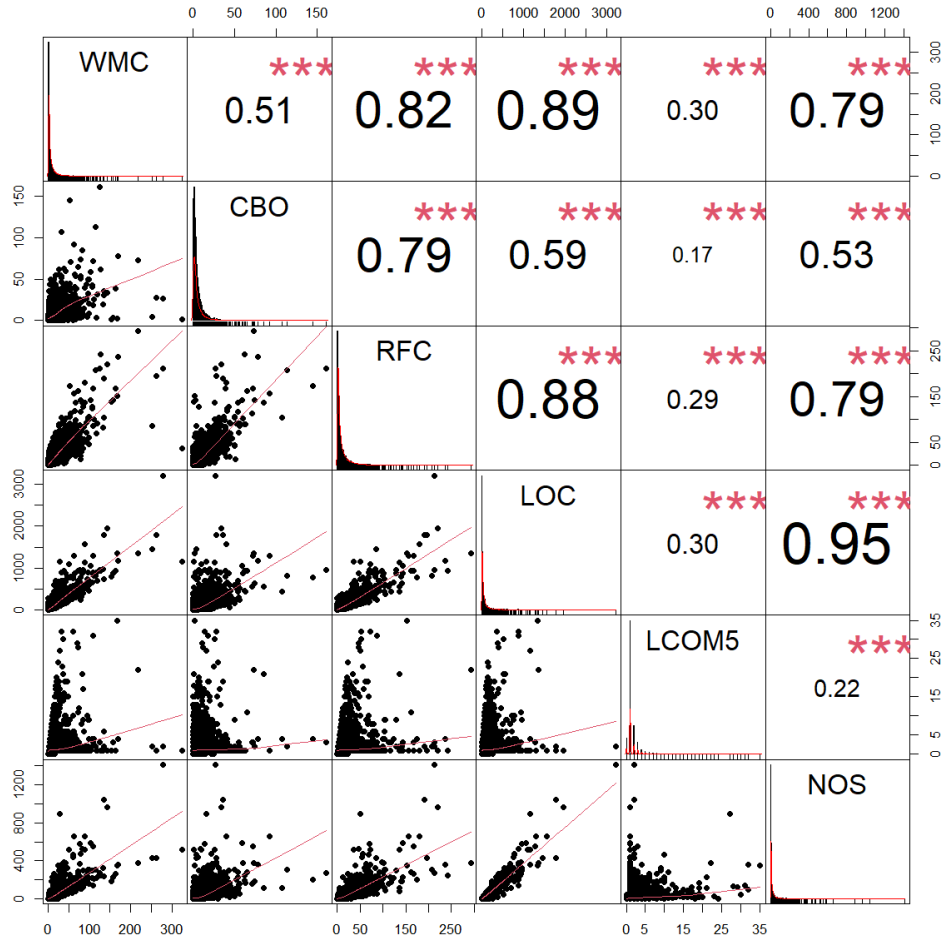


Figura 8: chart.Correlation — métricas principais (Class-level)

A Figura 8, gerada com o pacote PerformanceAnalytics, combina histogramas (diagonal), gráficos de dispersão (triângulo inferior) e coeficientes de correlação (triângulo superior), proporcionando uma visão integrada das relações entre variáveis.

Para o dataset File-level, a correlação mais forte é McCC $\times$ LLOC ($r = 0,72$), confirmando que a complexidade ciclomática cresce com o tamanho do código. A correlação entre “Number of previous modifications” e “Number of committers” também é forte ($r = 0,77$), indicando que arquivos modificados por mais desenvolvedores acumulam mais modificações.

Matriz de Correlação - File-level

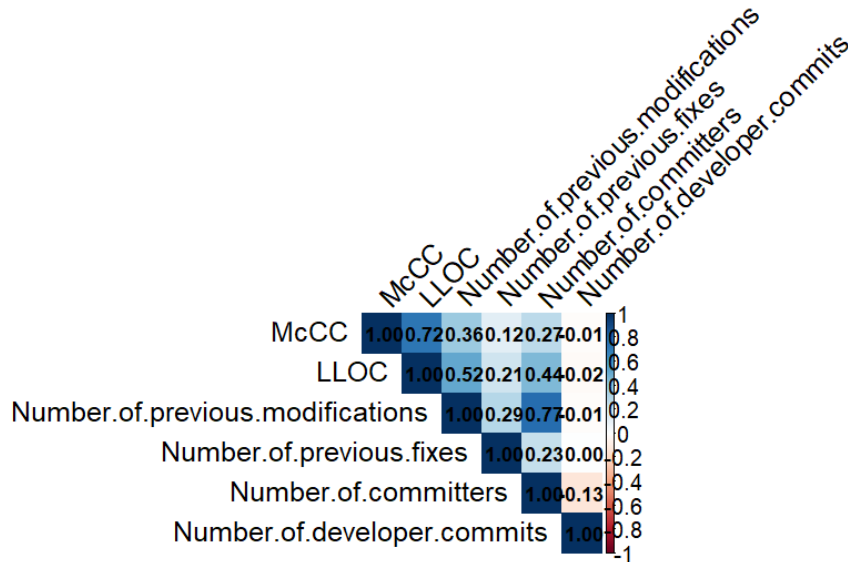


Figura 9: Heatmap de correlação — File-level

3.8 Modelagem Estatística — Regressão Logística

Para a modelagem, optou-se pela regressão logística binária, dada a natureza binária da variável alvo (Bug_class: 0 ou 1), derivada de “Number of bugs”.

3.8.1 Modelo Class-level

O modelo ajustado utiliza como preditores: WMC, CBO, RFC, LOC, LCOM5, NL, DIT e CC. A Tabela 6 apresenta os coeficientes.

Tabela 6: Regressão logística — Class-level

Variável	Odds Ratio	p -valor	Sig.
(Intercept)	0,0019	$3,77 \times 10^{-19}$	***
WMC	0,9920	0,8130	
CBO	1,0208	0,4335	
RFC	1,0270	0,2951	
LOC	1,0005	0,8747	
LCOM5	0,6512	0,3891	
NL	1,0325	0,8889	
DIT	0,5571	0,2098	
CC	≈ 0	0,3587	

Nenhum preditor individualmente alcançou significância estatística ($p < 0,05$). O modelo apresentou:

- AIC = 129,26
- Pseudo R^2 (McFadden) = 0,1194
- Acurácia = 99,90%

A alta acurácia é enganosa: o modelo classificou todas as instâncias como “sem bug”, o que já fornece 99,90% de acurácia devido ao desbalanceamento extremo. A matriz de confusão confirma que nenhuma das 8 instâncias com bug foi corretamente identificada.

3.8.2 Modelo File-level

Para o File-level, os preditores foram: McCC, LLOC, Number of previous modifications, Number of previous fixes, Number of committers e Number of developer commits.

Tabela 7: Regressão logística — File-level

Variável	Odds Ratio	p -valor	Sig.
(Intercept)	0,0009	$2,14 \times 10^{-12}$	***
McCC	0,9961	0,8844	
LLOC	1,0007	0,7612	
N. previous mods	1,0111	0,1695	
N. previous fixes	1,0574	0,9481	
N. committers	1,1120	0,3389	
N. dev commits	0,9993	0,1560	

O modelo File-level apresentou comportamento análogo: AIC = 90,13, Pseudo R^2 = 0,1614 e acurácia de 99,86%, mas sem capacidade de identificar os 6 arquivos com bugs. A variável “Number of previous modifications” apresentou o menor p -valor (0,1695), sugerindo uma tendência (não significativa) de associação com defeitos.

3.9 Discussão Geral

Padrões encontrados: As métricas de código-fonte do Neo4j seguem distribuições fortemente assimétricas, com a maioria das classes apresentando baixa complexidade e poucas concentrando valores extremos. Esse padrão é consistente com a literatura sobre métricas de software [7, 3].

Relações entre métricas: As correlações fortes entre LOC, LLOC, NOS, WMC e RFC indicam que essas métricas capturam dimensões sobrepostas do tamanho e complexidade do código. Em contextos de predição, essa multicolinearidade sugere que um subconjunto reduzido de métricas pode ser suficiente.

Métricas e defeitos: A correlação extremamente fraca entre todas as métricas e a variável de bugs (máximo $r = 0,06$ para CBO) contrasta com achados de estudos anteriores [1, 3]. Essa discrepância pode ser explicada pelo desbalanceamento extremo (apenas 8 e 6 instâncias com bugs nos datasets Class e File, respectivamente) e pela possibilidade de subreporte de defeitos no dataset.

Limitações da análise:

- O desbalanceamento severo ($< 0,15\%$ de instâncias positivas) compromete a modelagem estatística e impede conclusões sobre a associação entre métricas e defeitos;
- A coluna CLOC no dataset File-level contém apenas zeros, sugerindo falha na extração dessa métrica;
- O teste de Kolmogorov-Smirnov reportou *ties* nos dados, indicando que a versão utilizada assume distribuições contínuas;
- A regressão logística sem tratamento de desbalanceamento (e.g., SMOTE, pesos) tende a classificar tudo como a classe majoritária.

4 Conclusão

Este trabalho apresentou uma análise estatística completa de métricas de código-fonte do framework Neo4j, utilizando os datasets Class-level e File-level do GitHub Bug Dataset v1.1. A análise revelou que:

1. Todas as métricas de código-fonte seguem distribuições fortemente assimétricas à direita, com alta concentração de valores baixos e caudas longas, refletindo a estrutura típica de projetos de software orientado a objetos de grande porte.
2. Existem correlações fortes entre métricas de tamanho e complexidade ($\text{LOC} \times \text{LLOC}$: $r = 0,99$; $\text{WMC} \times \text{LOC}$: $r = 0,89$), indicando redundância e sugerindo que um subconjunto reduzido de métricas pode ser representativo.
3. Nenhuma variável segue distribuição normal, rejeitando-se a hipótese de normalidade com p -valores extremamente baixos em todos os testes de Shapiro-Wilk e Kolmogorov-Smirnov.
4. A proporção de outliers pelo método IQR varia entre 5% e 22%, sendo os outliers representativos de componentes legitimamente complexos do framework.

5. Os modelos de regressão logística não conseguiram prever instâncias com defeitos devido ao desbalanceamento extremo da variável alvo, limitação inerente ao dataset utilizado.

Como trabalhos futuros, sugere-se a aplicação de técnicas de balanceamento de classes (SMOTE, *undersampling*), a utilização de modelos não paramétricos (e.g., *Random Forest*) e a análise de versões anteriores do framework para obter maior representatividade de defeitos.

Referências

- [1] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering*, 22(10):751–761, 1996.
- [2] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.
- [3] Tibor Gyimóthy, Rudolf Ferenc, and István Siket. Empirical validation of object-oriented metrics on open source software for fault prediction. *IEEE Transactions on Software Engineering*, 31(10):897–910, 2005.
- [4] Nachiappan Nagappan, Thomas Ball, and Andreas Zeller. Mining metrics to predict component failures. *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, pages 452–461, 2006.
- [5] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill Education, 8th edition, 2014.
- [6] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025.
- [7] Danijel Radjenović, Marjan Heriko, Richard Torkar, and Aleš Živkovič. Software fault prediction metrics: A systematic literature review. *Information and Software Technology*, 55(8):1397–1418, 2013.
- [8] Samuel S. Shapiro and Martin B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3/4):591–611, 1965.
- [9] Zoltán Tóth, Pter Gyimési, and Rudolf Ferenc. A public bug database of GitHub projects and its application in bug prediction. In *Proceedings of the International Conference on Computational Science and Its Applications (ICCSA)*, pages 625–638. Springer, 2016.